PEP 735 – Dependency Groups in pyproject.toml

Important

This PEP is a historical document. The up-to-date, canonical spec, Dependency Groups, is maintained on the PyPA specs page.

See the PyPA specification update process for how to propose changes.

Abstract

This PEP specifies a mechanism for storing package requirements in pyproject.toml files such that they are not included in any built distribution of the project.

This is suitable for creating named groups of dependencies, similar to requirements.txt files, which launchers, IDEs, and other tools can find and identify by name.

The feature defined here is referred to as “Dependency Groups”.

Motivation

There are two major use cases for which the Python community has no standardized answer:

- How should development dependencies be defined for packages?
- How should dependencies be defined for projects which do not build distributions (non-package projects)?

In support of these two needs, there are two common solutions which are similar to this proposal:

- requirements.txt files
- package extras

Both requirements.txt files and extras have limitations which this standard seeks to overcome.

Note that the two use cases above describe two different types of projects which this PEP seeks to support:

- Python packages, such as libraries
- non-package projects, such as data science projects

Several motivating use cases are defined in detail in the Use Cases Appendix.

Limitations of requirements.txt files

Many projects may define one or more requirements.txt files, and may arrange them either at the project root (e.g. requirements.txt and test-requirements.txt) or else in a directory (e.g. requirements/base.txt and requirements/test.txt). However, there are major issues with the use of requirements files in this way:

- There is no standardized naming convention such that tools can discover or use these files by name.
- requirements.txt files are not standardized, but instead provide options to pip.

As a result, it is difficult to define tool behaviors based on requirements.txt files. They are not trivial to discover or identify by name, and their contents may contain a mix of package specifiers and additional pip options.

The lack of a standard for requirements.txt contents also means they are not portable to any alternative tools which wish to process them other than pip.

Additionally, requirements.txt files require a file per dependency list. For some use-cases, this makes the marginal cost of dependency groupings high, relative to their benefit. A terser declaration is beneficial to projects with a number of small groups of dependencies.

In contrast with this, Dependency Groups are defined at a well known location in pyproject.toml with fully standardized contents. Not only will they have immediate utility, but they will also serve as a starting point for future standards.

Limitations of extras

extras are additional package metadata declared in the [project.optional-dependencies] table. They provide names for lists of package specifiers which are published as part of a package's metadata, and which a user can request under that name, as in pip install 'foo[bar]' to install foo with the bar extra.

Because extras are package metadata, they are not guaranteed to be statically defined and may require a build system to resolve. Furthermore, definition of a [project.optional-dependencies] indicates to many tools that a project is a package, and may drive tool behaviors such as validation of the [project] table.

For projects which are packages, extras are a common solution for defining development dependencies, but even under these circumstances they have downsides:

- Because an extra defines optional additional dependencies, it is not possible to install an extra without installing the current package and its dependencies.
- Because they are user-installable, extras are part of the public interface for packages. Because extras are published, package developers often are concerned about ensuring that their development extras are not confused with user-facing extras.

Rationale

This PEP defines the storage of requirements data in lists within a [dependency-groups] table. This name was chosen to match the canonical name of the feature (“Dependency Groups”).

This format should be as simple and learnable as possible, having a format very similar to existing requirements.txt files for many cases. Each list in [dependency-groups] is defined as a list of package specifiers. For example:

```
[dependency-groups]
test = ["pytest>7", "coverage"]
```

There are a number of use cases for requirements.txt files which require data which cannot be expressed in PEP 508 dependency specifiers. Such fields are not valid in Dependency Groups. Including many of the data and fields which pip supports, such as index servers, hashes, and path dependencies, requires new standards. This standard leaves room for new standards and developments, but does not attempt to support all valid requirements.txt contents.

The only exception to this is the -r flag which requirements.txt files use to include one file in another. Dependency Groups support an “include” mechanism which is similar in meaning, allowing one dependency group to extend another.

Dependency Groups have two additional features which are similar to requirements.txt files:

- they are not published as distinct metadata in any built distribution
- installation of a dependency group does not imply installation of a package’s dependencies or the package itself

Use Cases

The following use cases are considered important targets for this PEP. They are defined in greater detail in the Use Cases Appendix.

- Web Applications deployed via a non-python-packaging build process
- Libraries with unpublished dev dependency groups
- Data science projects with groups of dependencies but no core package
- Input data to lockfile generation (Dependency Groups should generally not be used as a location for locked dependency data)
- Input data to an environment manager, such as tox, Nox, or Hatch
- Configurable IDE discovery of test and linter requirements